



UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MÉXICO

FACULTAD DE INGENIERÍA

Sistemas Embebidos

Diseño y Arquitectura: Plataforma de Registro de Pedidos

Docente:
Jaime García García

Alumnos:

Rodrigo Mejía Fabila

Diego Alcántara Ortega

1. Planteamiento y Alcance

El sistema por desarrollar es una plataforma de registro de pedidos diseñada para operar en un entorno distribuido. El objetivo es evaluar el comportamiento del sistema bajo operaciones concurrentes, midiendo su rendimiento, escalabilidad y tolerancia a fallos mediante la orquestación de contenedores. La plataforma procesará cargas de trabajo simuladas, validando la integridad de los datos y la resiliencia de las réplicas ante interrupciones.

2. Arquitectura del Sistema

El sistema se compone de una arquitectura en tres capas principales:

- **API REST y Procesador (Python + FastAPI):** Un único microservicio unificado que recibe peticiones, valida los datos de entrada, simula un tiempo de procesamiento y registra la operación. Está diseñado para admitir múltiples réplicas concurrentes.

- **Persistencia (PostgreSQL):** Base de datos relacional encargada de almacenar las operaciones de forma segura y verificar las reglas de integridad de los datos bajo peticiones simultáneas.

- **Infraestructura (Docker y Kubernetes):** Los servicios se empaquetarán en imágenes de Docker y se desplegarán mediante manifiestos YAML en un clúster local administrado por Minikube.

- **Generador de Carga (Locust / k6):** Herramienta externa para inyectar tráfico concurrente y extraer las métricas de prueba.

3. Modelo de Datos

La base de datos contará con una tabla principal para garantizar la persistencia de las operaciones:

- **Tabla: pedidos**

- **Columnas:**

    - id (UUID, Primary Key)
    - producto (String, Not Null)
    - cantidad (Integer, Not Null)
    - estado (String, Default: 'procesado')
    - fecha_creacion (Timestamp)

4. Definición de la API

El microservicio expondrá los siguientes endpoints:

- **POST /pedidos:** Recibe un payload JSON con producto y cantidad. Simula una carga de CPU breve, inserta el registro en PostgreSQL y devuelve un código HTTP 201 (Created).
- **GET /pedidos:** Devuelve una lista de los pedidos recientes o el conteo total para verificar la integridad de los registros insertados.
- **GET /health:** Endpoint ligero utilizado por Kubernetes para los *liveness* y *readiness probes* para saber si el pod está funcionando.

5. Diseño Experimental, Métricas e Hipótesis

El sistema será sometido a cinco escenarios de prueba (Línea base, Escalabilidad, Concurrencia, Fallo de instancia y Persistencia). Se observarán las siguientes métricas:

- **Rendimiento:** Solicitudes completadas por segundo (req/s).
- **Latencia:** Mediana y percentil 95 (p95) del tiempo de respuesta.
- **Tasa de error:** Porcentaje de solicitudes fallidas frente al total.
- **Hipótesis:** Al escalar el despliegue del procesador de 1 a 3 réplicas, la tasa de solicitudes completadas por segundo (req/s) aumentará significativamente bajo cargas altas de concurrencia, siempre y cuando la base de datos PostgreSQL no se convierta en el cuello de botella. La latencia p95 debería estabilizarse al distribuir la carga.